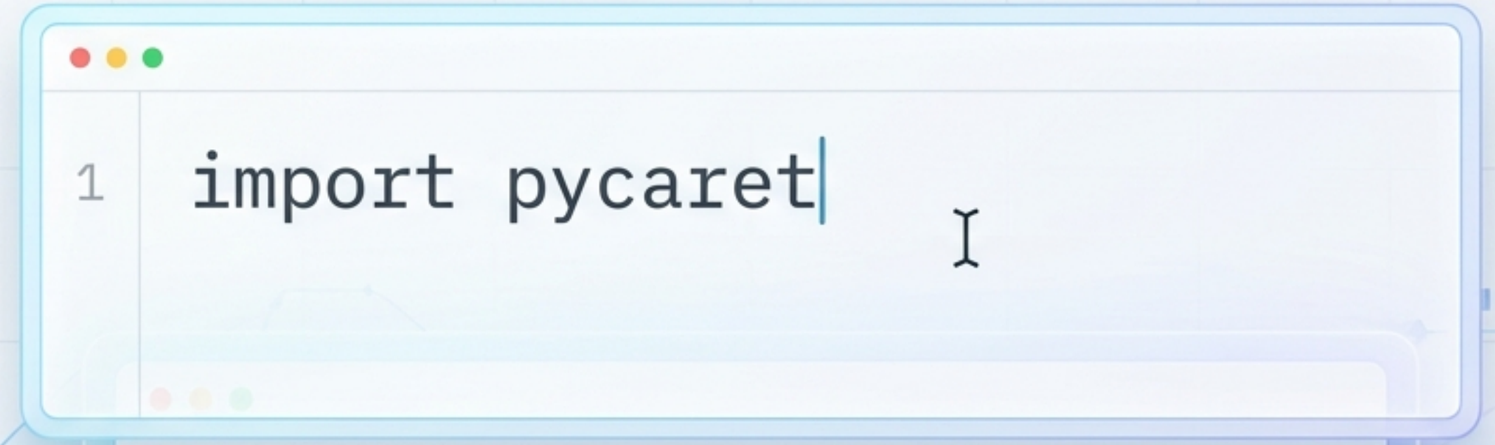
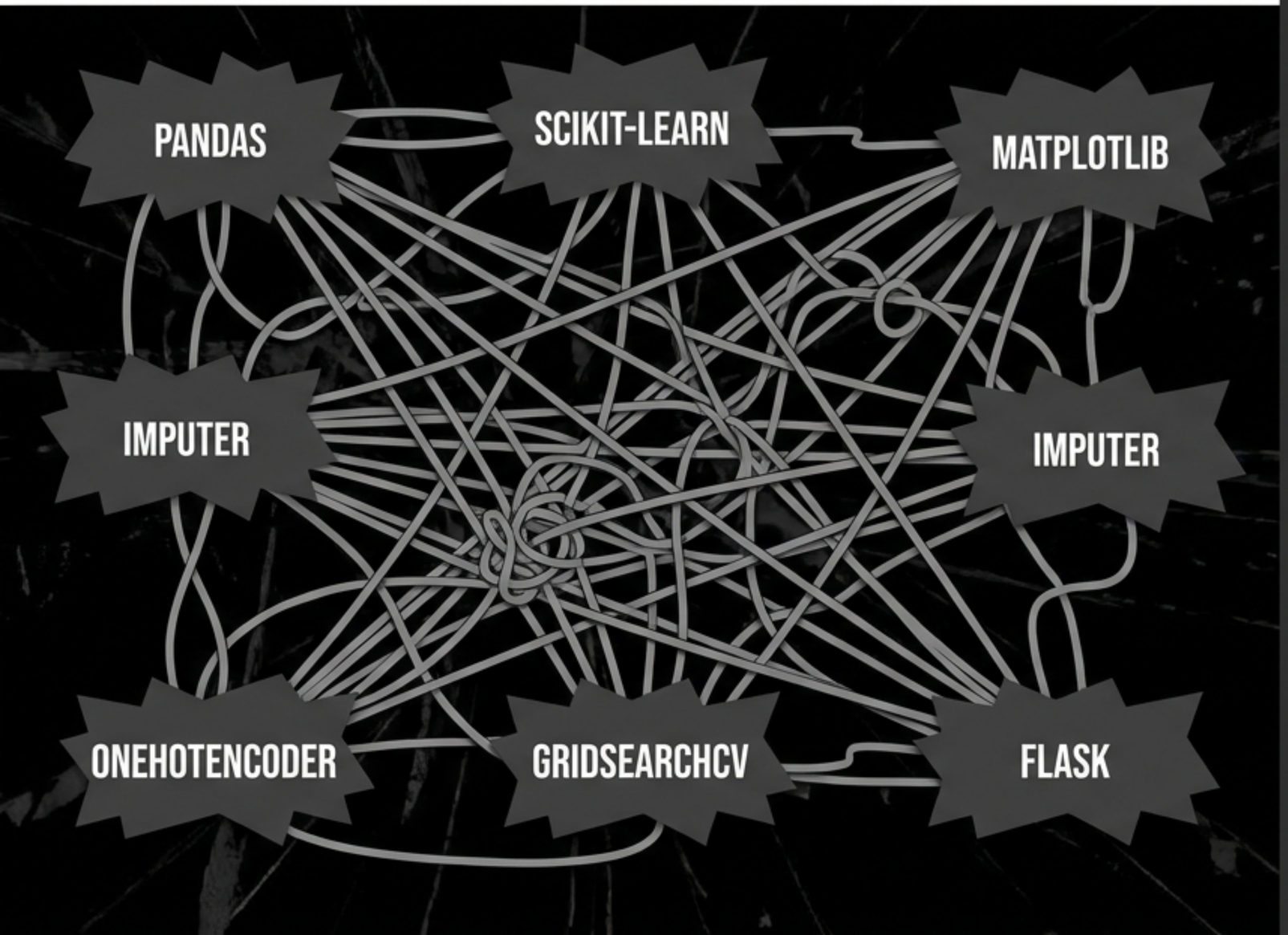


Machine Learning on Autopilot

Accelerating the ML Lifecycle with PyCaret's Low-Code Unified API.

THE TRADITIONAL ML BOTTLENECK

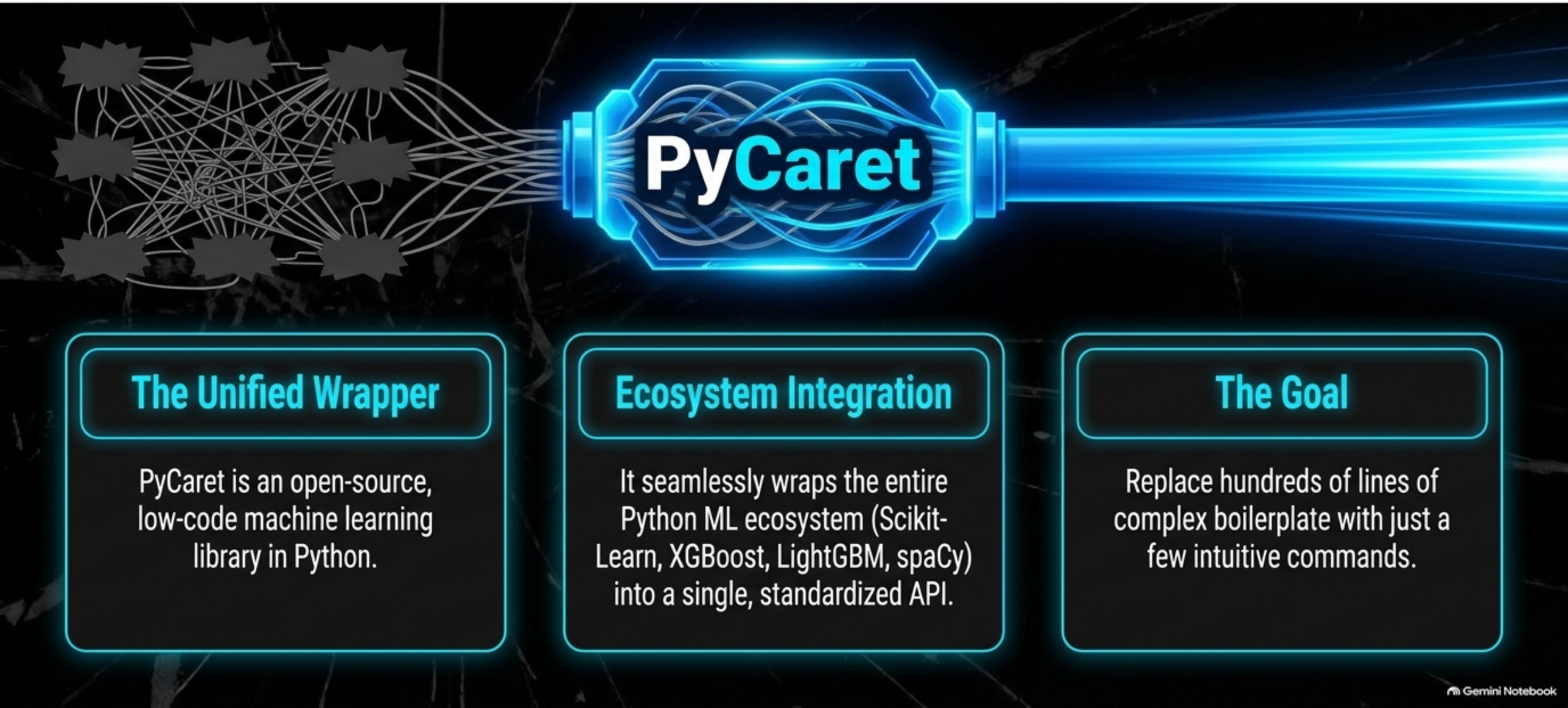


THE ITERATIVE TRAP

The traditional ML lifecycle (Problem -> **Data Prep** -> Model Dev -> Deploy) is **code-heavy** and **deeply fragmented** across dozens of independent libraries.

THE 80% RULE

Data scientists spend 80% of their time writing boilerplate code for data cleaning, visualization, and deployment rather than solving core business problems.



The Unified Wrapper

PyCaret is an open-source, low-code machine learning library in Python.

Ecosystem Integration

It seamlessly wraps the entire Python ML ecosystem (Scikit-Learn, XGBoost, LightGBM, spaCy) into a single, standardized API.

The Goal

Replace hundreds of lines of complex boilerplate with just a few intuitive commands.

The 5 Pillars of PyCaret

Classification	Regression	Clustering	Anomaly	Time Series
Supervised	Supervised	Unsupervised	Unsupervised	Forecasting
Categorical Target	Continuous Target	No Target	No Target	Temporal Target
Use Cases: Churn, Defaults, Medical Diagnosis	Use Cases: Price Prediction, Sales Forecasting	Use Cases: Customer Segmentation, Pattern Discovery	Use Cases: Fraud Detection, Fault Detection	Use Cases: Stock Prices, Demand Forecasting

The PyCaret Lifecycle Pipeline

One Pipeline, Any Algorithm

Every machine learning task in PyCaret follows the exact same chronological pipeline.

The Ultimate Abstraction

Learn the syntax once, and apply it effortlessly to any algorithm or data problem.

Step 1: The Setup Engine

setup() initializes the training environment and creates the transformation pipeline instantly. It handles imputation, encoding, target balancing (SMOTE), outlier removal, and train-test splits.

```
setup(data, target='price', normalize=True, pca=True)
```

Step 2: The Ultimate Shortcut

Compare_models Leaderboard:

Why guess the best algorithm? Train them all simultaneously.

compare_models() trains and evaluates all available models in the library using cross-validation and sorts them by your chosen metric.

Model	Accuracy	AUC	Recall
XGB	**0.94**	**0.98**	**0.92**
LGBM	0.92	0.97	0.90
RF	0.89	0.95	0.87
SVM	0.85	0.93	0.83
KNN	0.82	0.91	0.80

```
best_model = compare_models(sort='AUC')
```

Step 3: Model Creation & Optimization

Isolate & Train

Instantiates and trains a specific model with default hyperparameters using cross-validation.

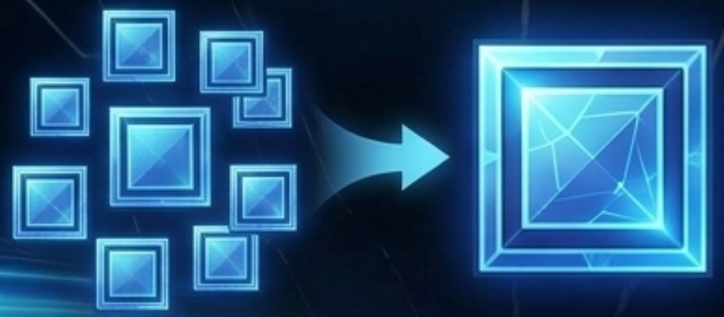
```
rf = create_model('rf')
```

Auto-Optimize

Automatically optimizes hyperparameters using grid search or randomized search libraries to maximize a specific metric.

```
tuned_rf = tune_model(rf, optimize='Accuracy')
```

Step 4: Advanced Architectures



Ensemble: `ensemble_model()` applies bagging or boosting to a single base estimator.

Blend: `blend_models()` uses soft or hard voting across diverse models.

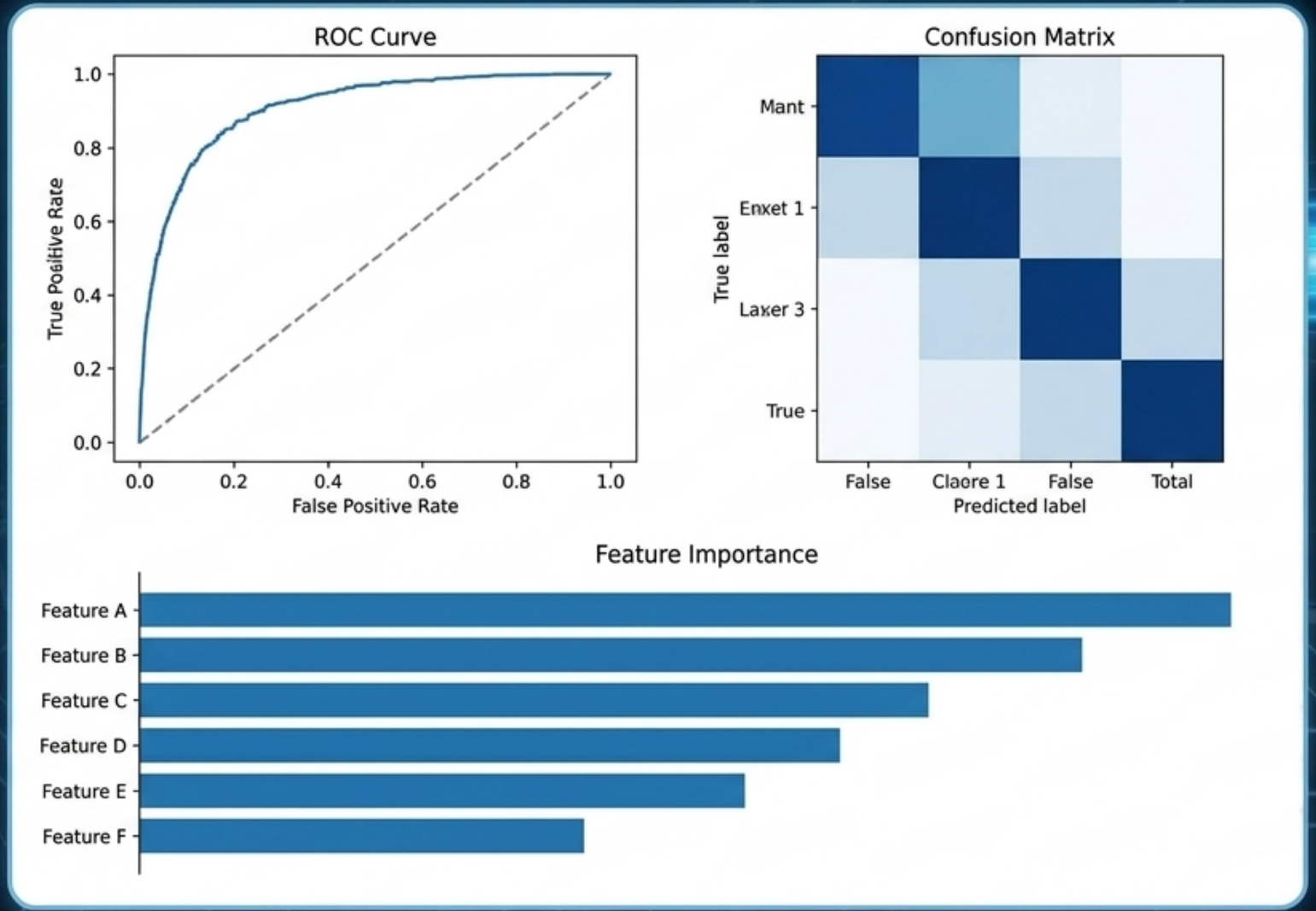
Stack: `stack_models()` uses predictions of base models as input features for a final meta-classifier.

Step 5: Visual Evaluation

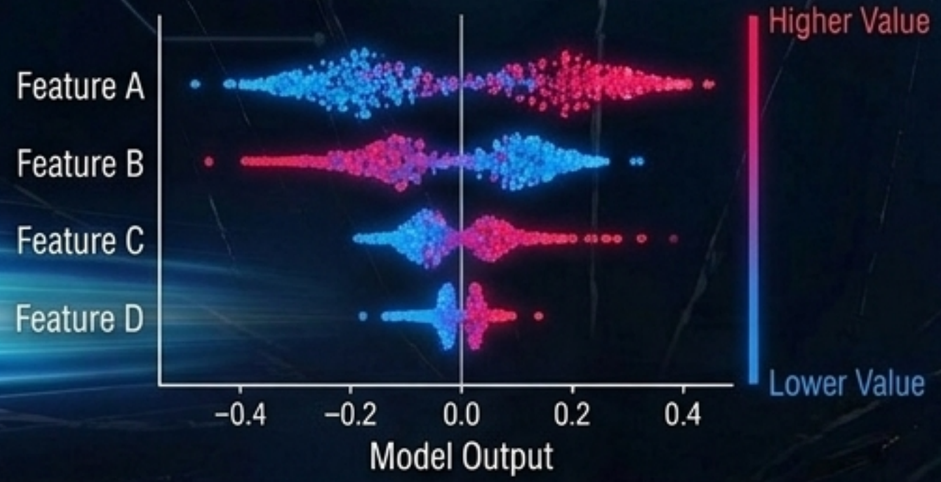
Leave Matplotlib behind. Generate production-ready diagnostic plots instantly.

evaluate_model() spawns an interactive UI dashboard directly in Jupyter notebooks notebooks to click through all available plots.

```
plot_model(rf, plot='feature')
```



Interpretability & Fairness

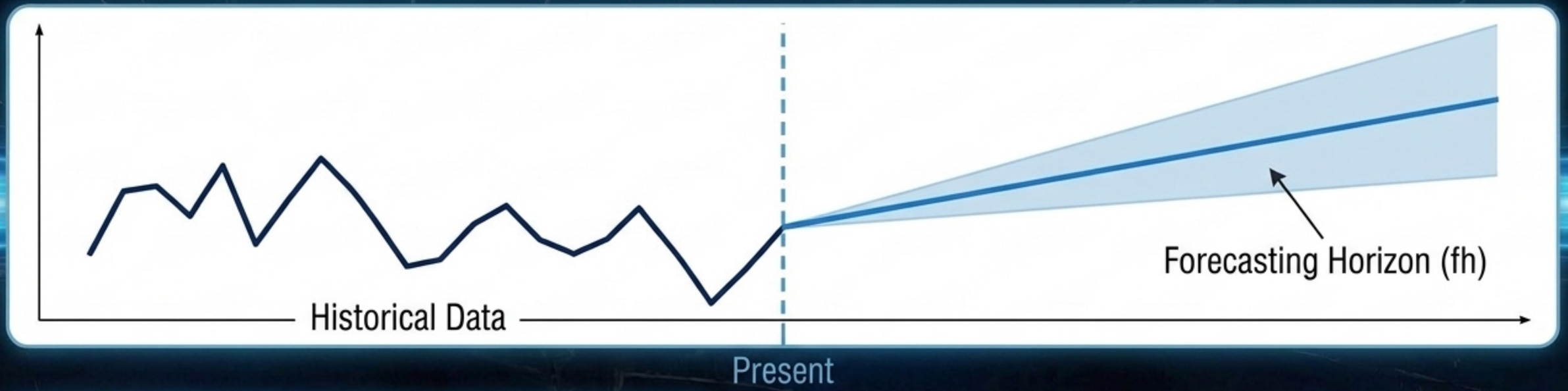


Unpacking the Black Box: `interpret_model()` leverages SHAP game theory to explain individual predictions and global feature impacts.

FAIRNESS SCORE: OPTIMAL

Ethical AI: `check_fairness()` assesses model bias across sensitive demographic groups to ensure ethical deployments before production.

Time Series Specifics: Navigating Time

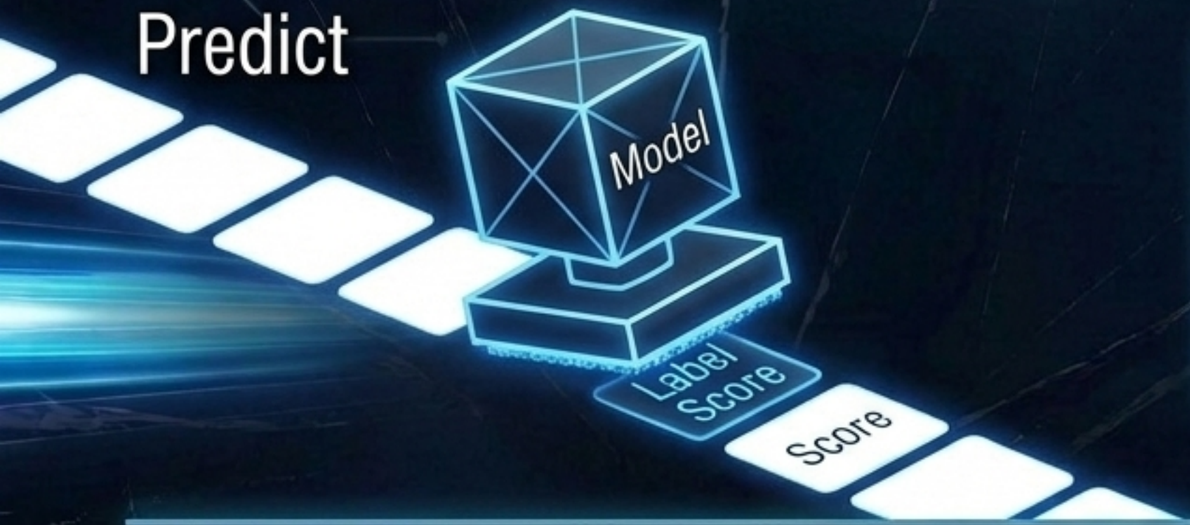


pycaret.time_series accommodates temporal complexity with automated seasonality detection and Expanding Window cross-validation.

```
setup(data, fh=12,
      fold_strategy='expanding')
```

Step 6: Finalization & Inference

Predict



predict_model(): Generates predictions and probabilities on unseen holdout data to verify final real-world performance.

Finalize

finalize_model(): Retrains the winning pipeline on the entire dataset—including the initial holdout split—to maximize data usage before production.

Step 7: From Notebook to Production

save_model(): Dumps the entire transformation pipeline and trained model as a reproducible Pickle file.

create_api() / create_docker(): Instantly generates a FastAPI backend and Dockerfile for the trained model.

deploy_model(): Pushes the packaged environment directly to AWS S3, GCP, or Microsoft Azure.

The PyCaret Unified API

Abstracting away the syntax complexity of Machine Learning. Focus on data, not syntax.

	Classification	Regression	Clustering	Anomaly	Time Series
Initialize	setup()	setup()	setup()	setup()	setup()
Train All	compare_models()	compare_models()	compare_models()	compare_models()	compare_models()
Train One	create_model()	create_model()	create_model()	create_model()	create_model()
Tune	tune_model()	tune_model()	tune_model()	tune_model()	tune_model()
Predict	predict_model()	predict_model()	predict_model()	predict_model()	predict_model()